Lab 3: Aerial Robotics

Group 6
Abdalrahman Nasser - abdnas-2@student.ltu.se
Addo Dominic - domadd-2@student.ltu.se
Aittokallio Deborah - debait-2@student.ltu.se
Bardia Baigy - barbai-1@student.ltu.se

Objective

The aim of this lab was to design, implement, and validate control strategies for the Crazyflie 2.0 quadcopter, later within a simulated environment due to time constraints. The specific goals included creating a stable hovering controller, implementing a position controller to move between poses (Pose A to Pose B), and following defined geometric trajectories (square, circle, and 45 degree tilted circle) using a cascaded PID control structure.

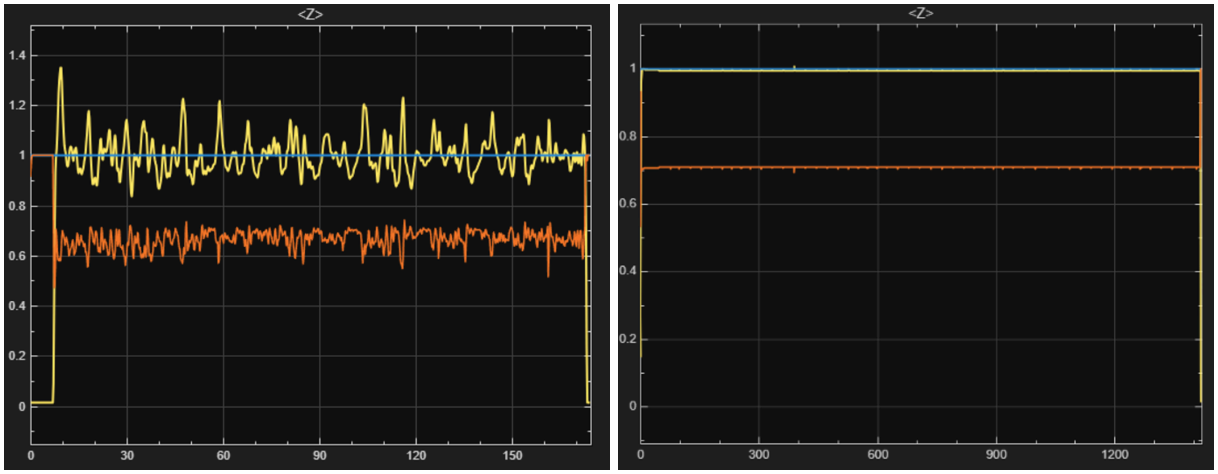
Methodology and system setup

Simulation Environment Configuration

A critical aspect of this project was tuning the simulation parameters to ensure stability. The default Gazebo settings in the provided ROS VM (500 Hz update rate, 0.002s step size) resulted in significant desynchronization and destabilization of the drone due to hardware latency in the virtual machine.
To counter this, we modified the physics engine configuration:

Real-Time Update Rate: Increased to 1000 Hz.
Max Step Size: Decreased to 0.001 s.

This adjustment lowered the Real Time Factor to approximately 0.6–0.7, meaning the simulation ran slower than wall-clock time. This tradeoff was necessary to guarantee the solver had sufficient time to compute the physics without "skipping" steps, which was essential for the stability of the drone's hover even before position control was applied.

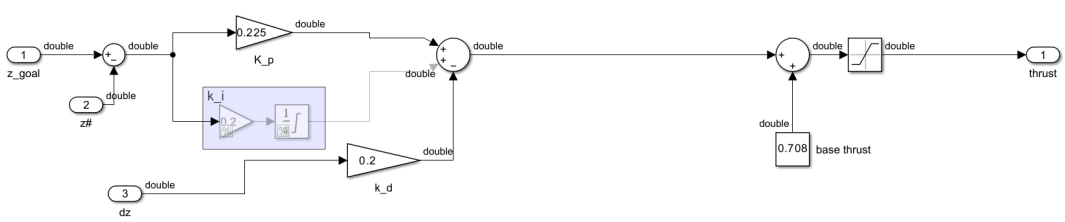


Graphs comparing the difference in height control between standard and lower step size in the Z axis

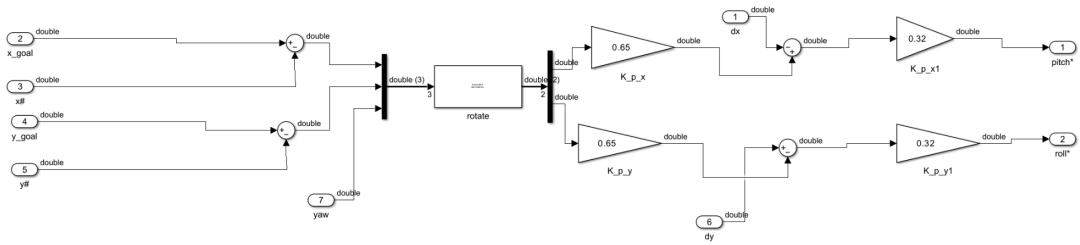
Control Architecture

Unlike standard implementations using pre-built PID blocks, we designed our controllers using individual logic blocks in Simulink to maximize modularity and tuning control.

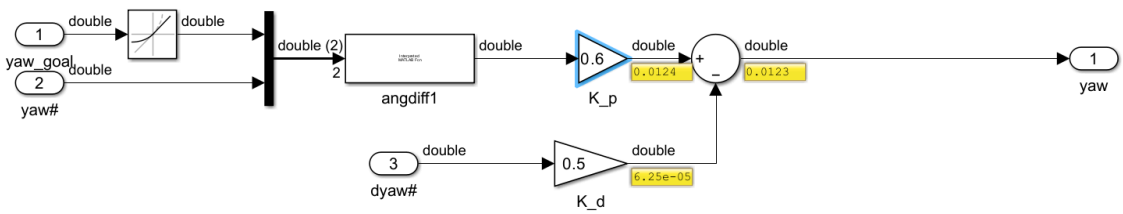
- **Altitude/hover Control:** We implemented a PID controller with a static **Base Thrust** feedforward term to counteract gravity. The Integral (I) term included saturation limits to prevent wind-up during takeoff which was useful in real life application but lead to more overshoot in the virtual simulation, most likely due to the desync problem.



- **Position Control (World-to-Body Transformation):** The position error is calculated in the World Frame. To translate this into Roll and Pitch commands (which operate in the Body Frame), we implemented a rotation matrix block (rotate). This block takes the current Yaw angle and rotates the World X/Y errors into the Body X/Y frame.



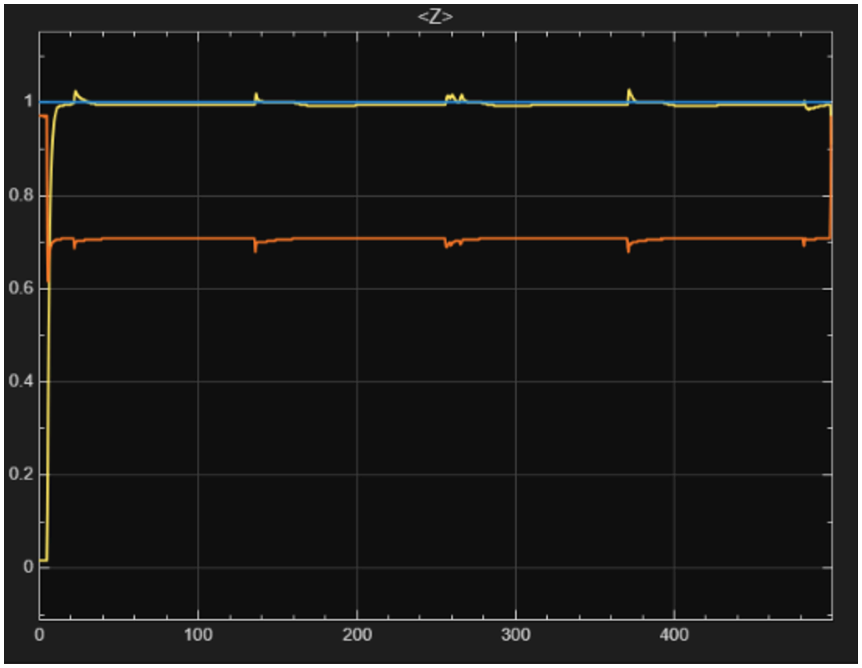
- **Yaw Control:** A PD controller regulates the heading. We incorporated an angdiff function to calculate the shortest angular distance to the target, preventing the drone from performing unnecessary 360° rotations when the error crossed the $\pm \pi$ boundary.



Results & Discussion

Task 1: Hovering Stability

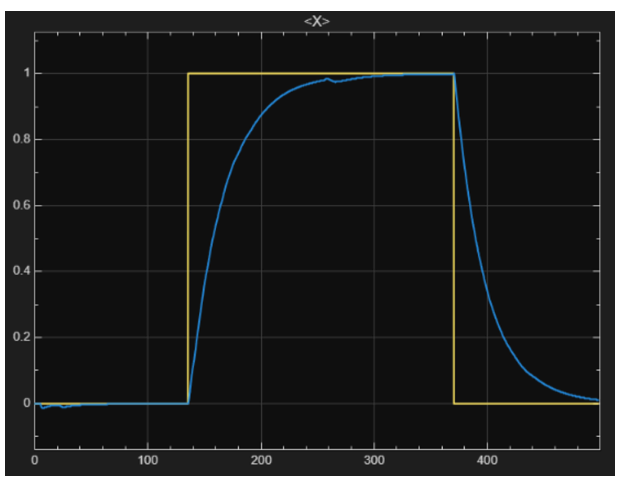
The altitude controller successfully maintained the Crazyflie at the reference height (1 m). Initial oscillations were dampened by tuning the Derivative (D) term and base thrust T_0 of the altitude controller. The main factor for success here was the simulation time-step reduction (0.001s), which eliminated the jitter observed at lower update rates.

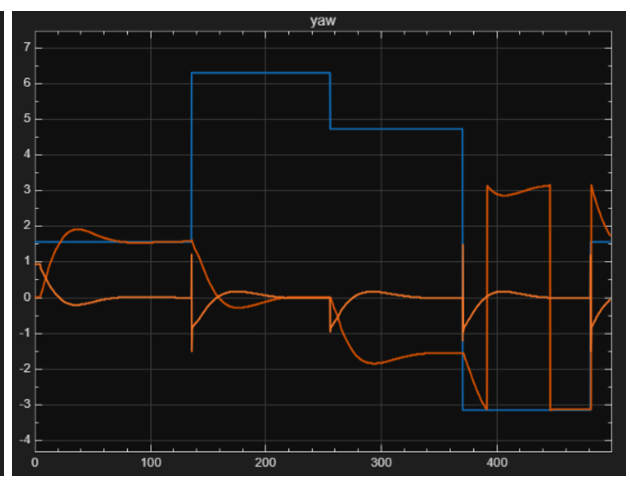


The blue line is the target hight, the yellow is the measured height, and the orange is the thrust signal
Note that the spikes are at the corners of the square flight path, otherwise the z error was smooth

Task 2: Pose to Pose & Manual Control

We validated the position controller by navigating between discrete poses (Pose A to Pose B). Additionally, we mapped manual inputs to the controller to verify robustness. The manual control worked flawlessly, allowing the operator to stabilize and move the drone intuitively. This step confirmed that the coordinate frame rotation logic was correct before attempting automated trajectories.

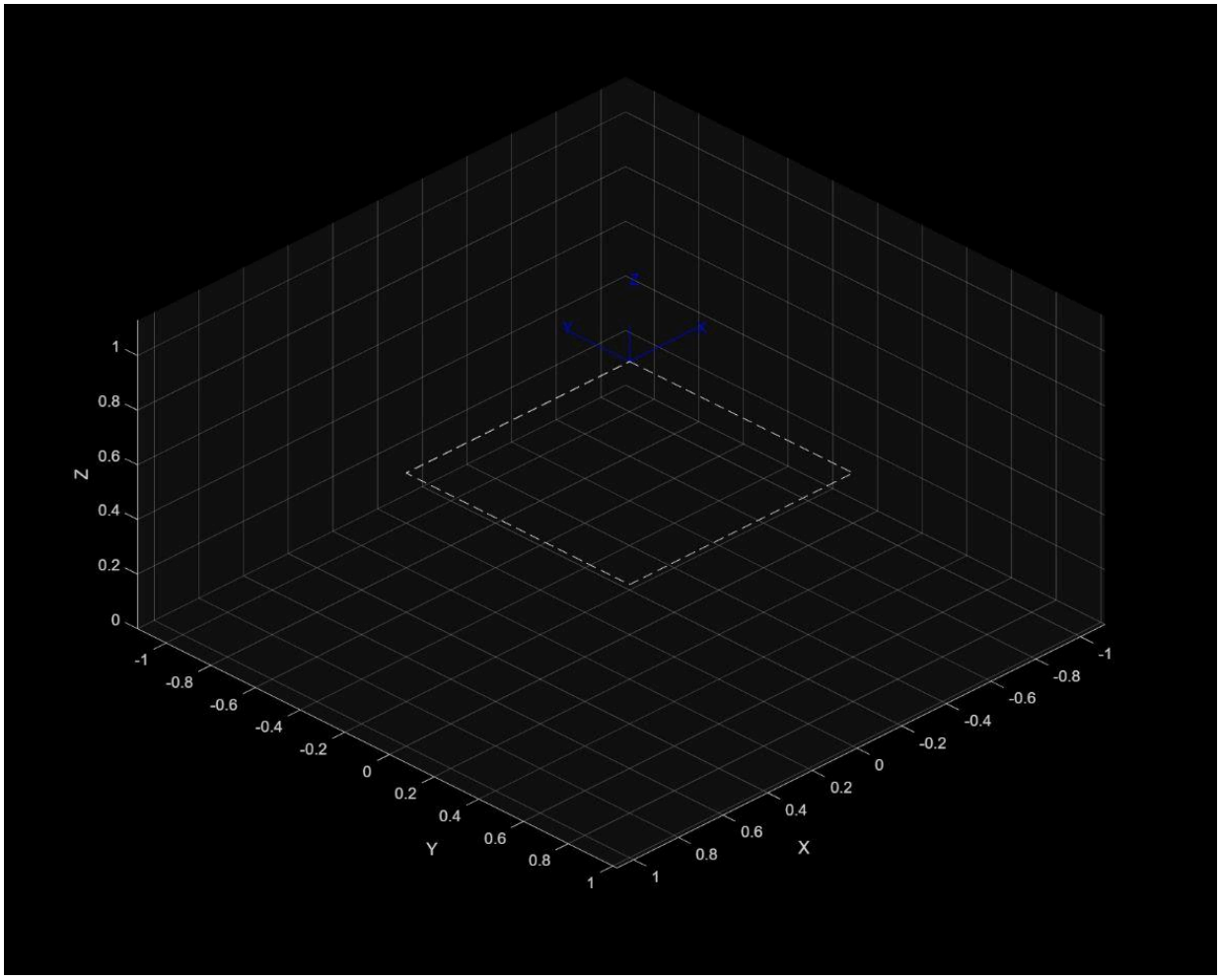


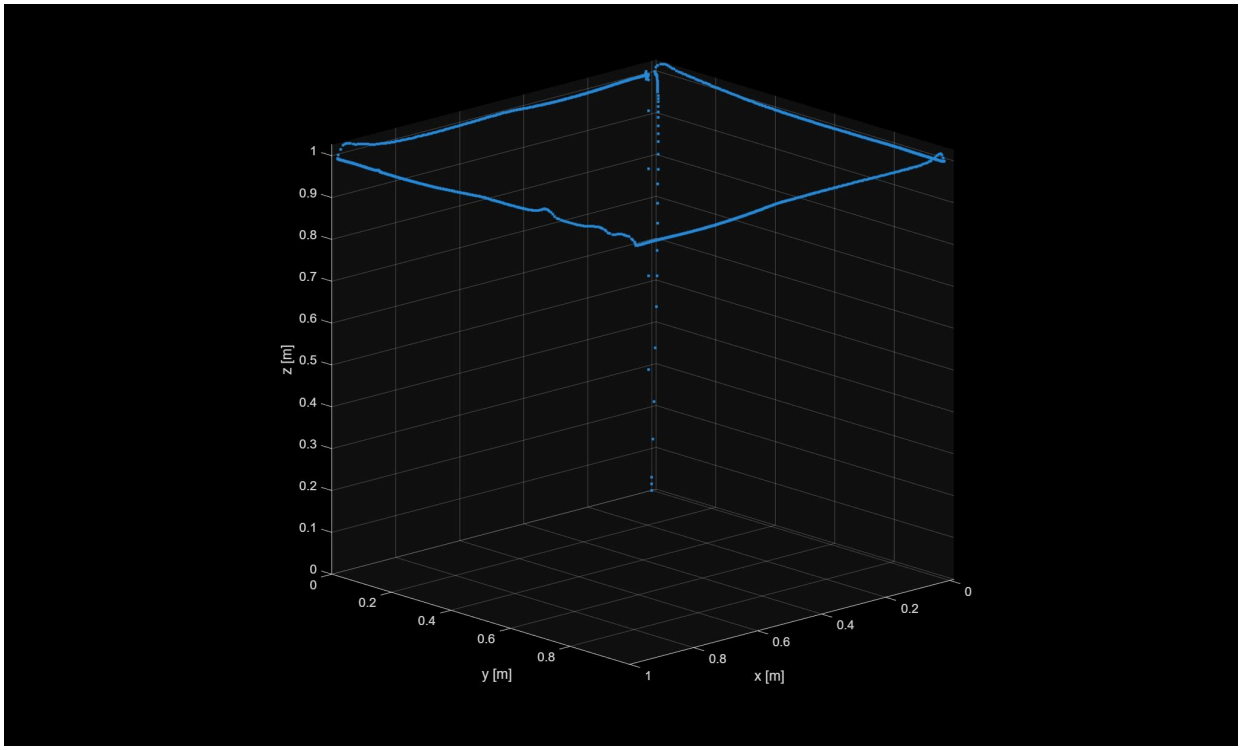


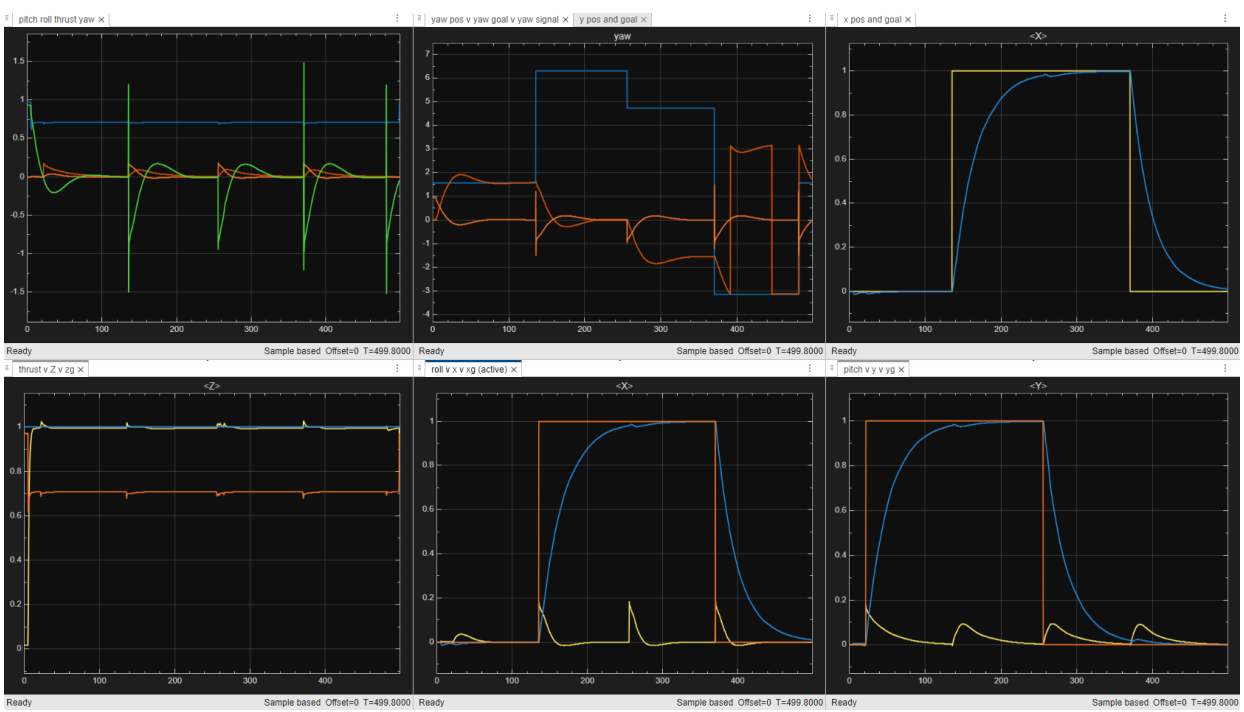
Task 3: Trajectory Tracking

3.1 Square Trajectory (Constant & Variable Yaw)

The drone followed the square path defined by waypoints. A notable issue encountered was Yaw-Induced Elevation, as the drone performed 90° turns at the corners of the square, the thrust vector shifted, causing a temporary spike in altitude. We implemented a compensation mechanism using a Sum block in the final thrust signal. This injected a corrective factor during aggressive yaw movements, stabilizing the altitude during turns.



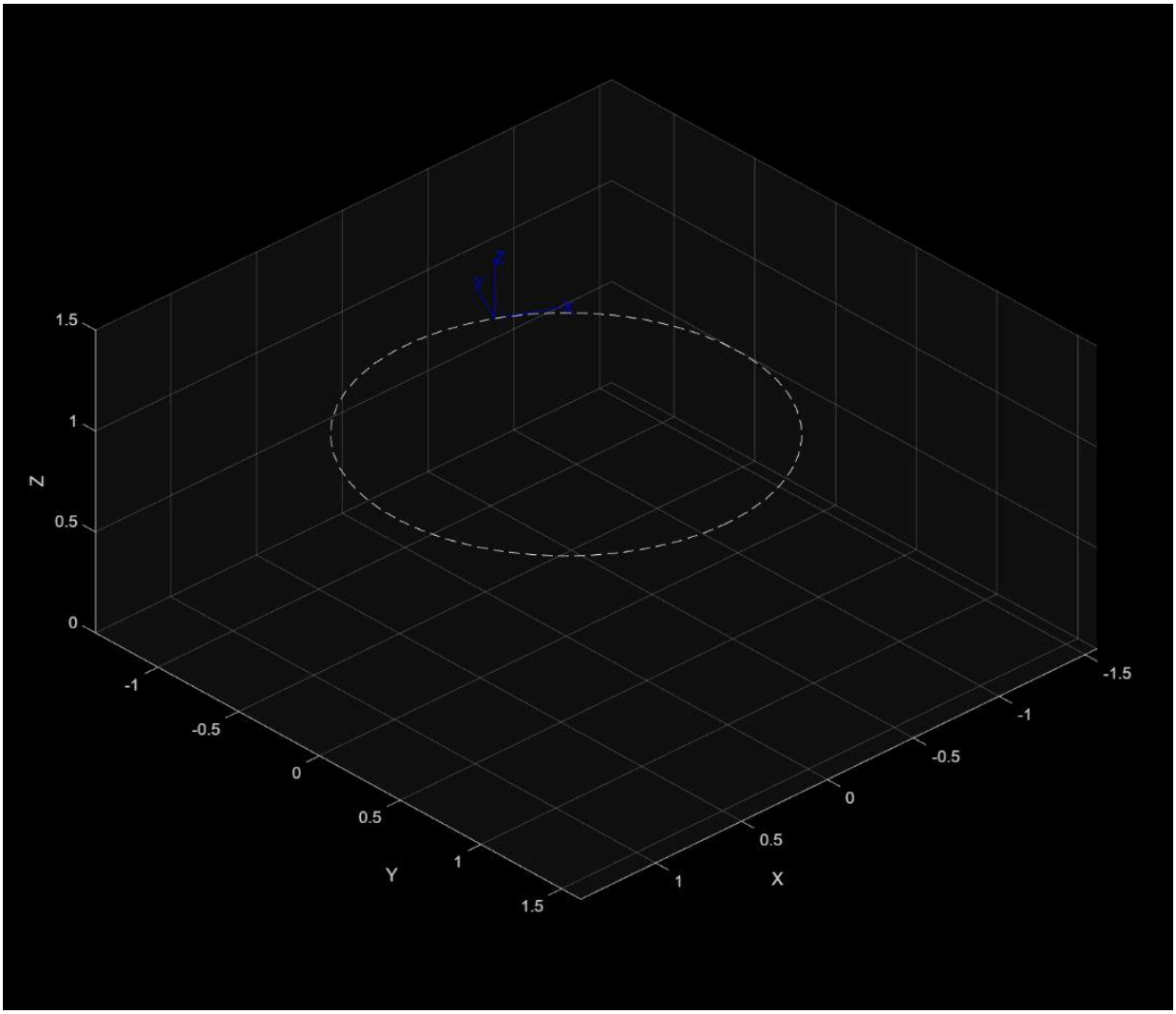


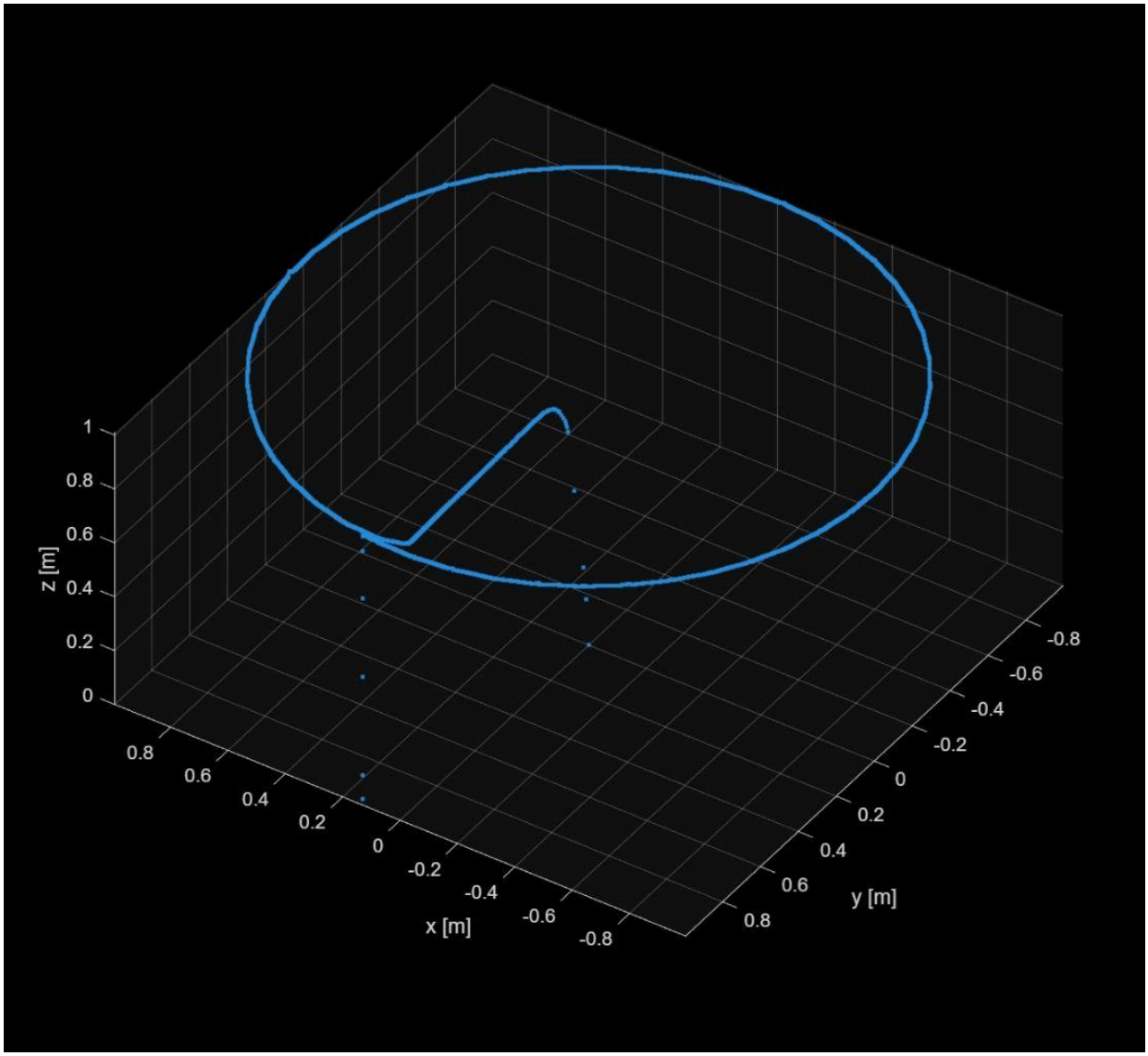


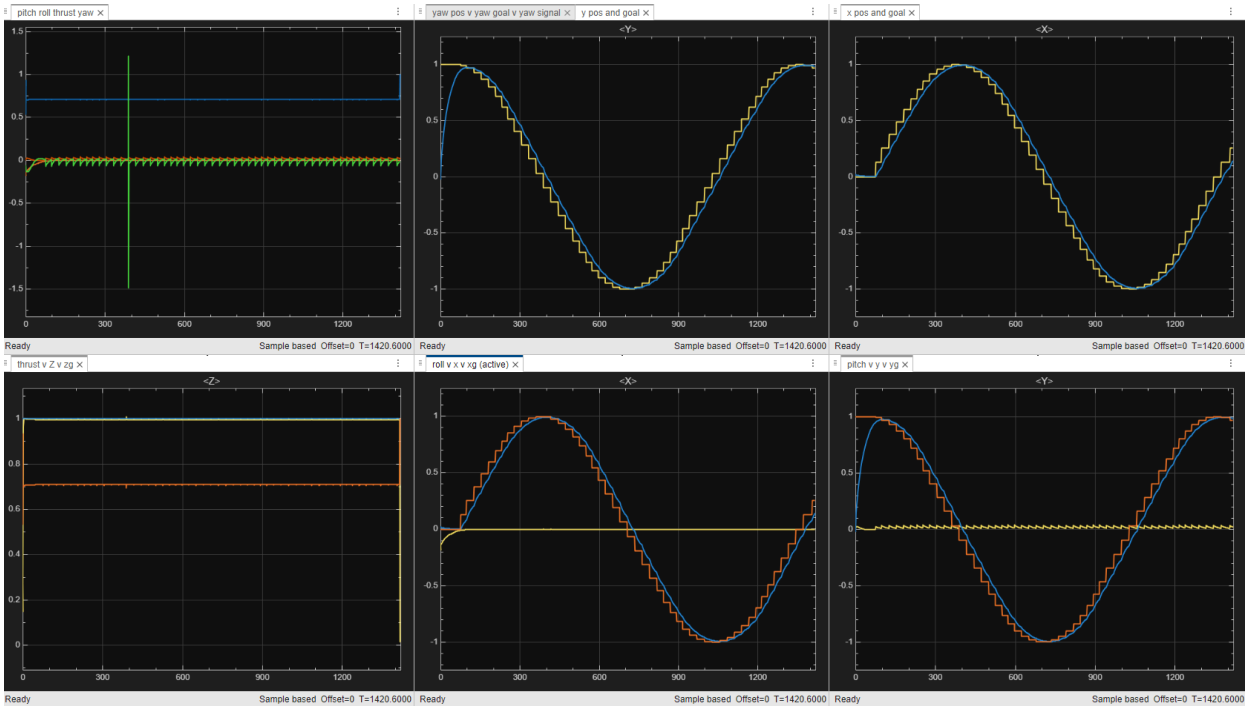
3.2 Horizontal Circle (Aiming Center)

For the standard horizontal circle (1m diameter), the drone tracked the path while continuously adjusting yaw to face the center.

We did not need to use the mstraj (Minimum Snap Trajectory) generator for this path. Because the simulation RTF was low (0.6x speed) and the drone only moved in the X-Y plane, the standard waypoint tracking was sufficiently smooth.



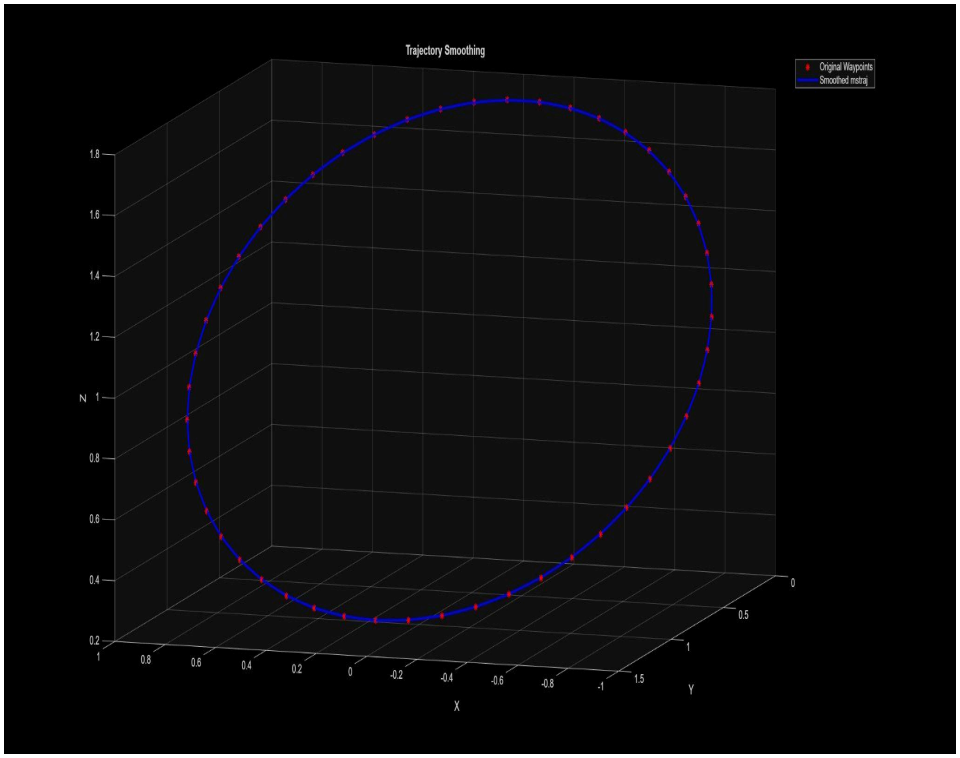

z [m]
1
0.8
0.6
0.4
0.2
0
x [m]
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
y [m]
-0.8
-0.6
-0.4
-0.2
0
0.2
0.4
0.6
0.8

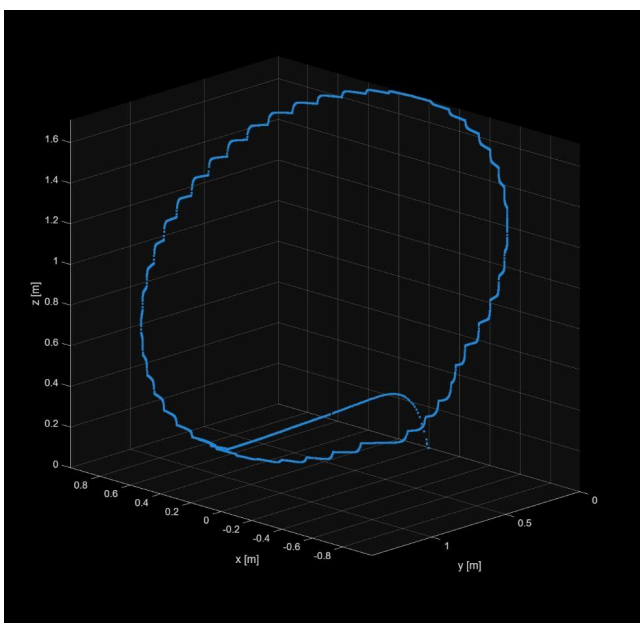


3.3 Tilted Circle (45 Degrees)

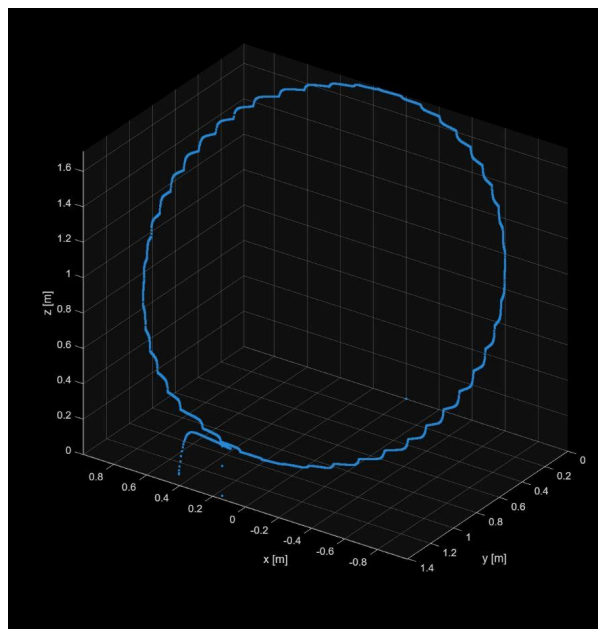
The tilted circle presented a unique challenge: unlike the horizontal circle, this path required rapid, continuous changes in the **Z-axis** (Altitude) synchronized with X and Y movement.

Without trajectory smoothing, the discrete waypoints caused jerky vertical motion, as the altitude controller struggled to track the staircase inputs of the raw tilted coordinates. We implemented mstraj to generate a smoothed path with velocity limits. This significantly improved the tracking performance. While the circle was not geometrically perfect due to the aggressive slope and limited tuning time, the motion was continuous and stable, improvements could be easily achievable with more segmented points.

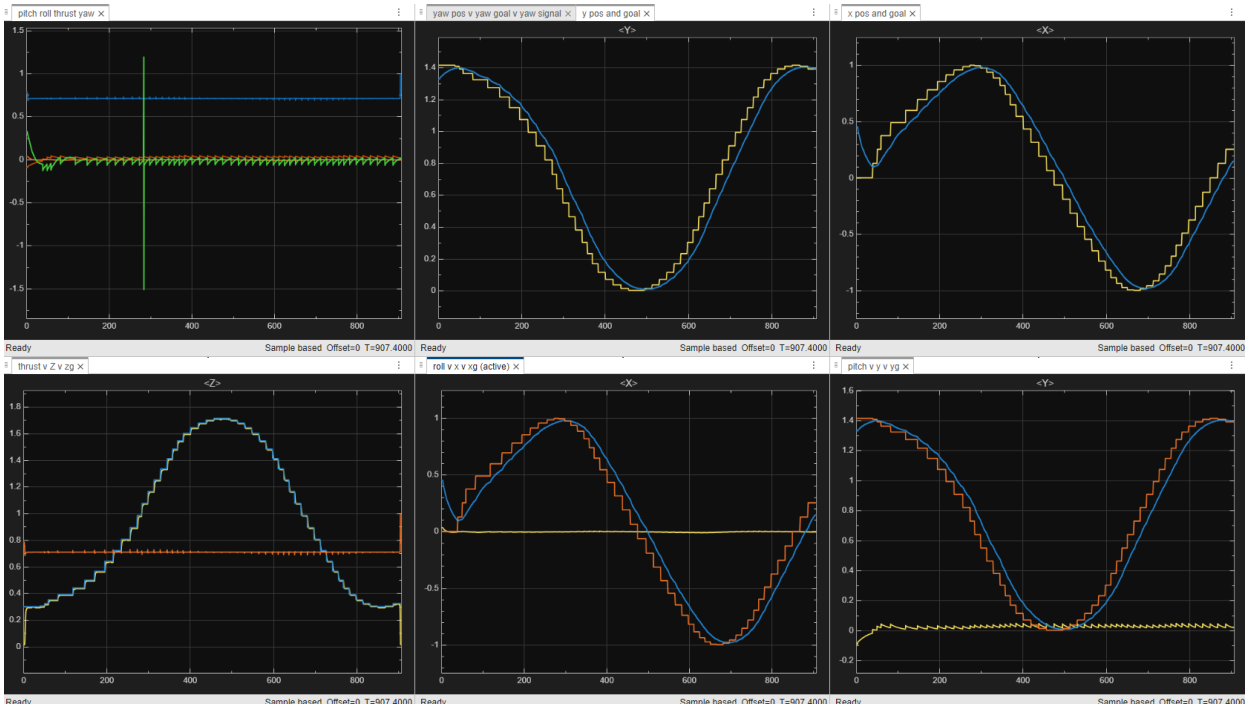




Flight path without mstraj



Flight path with mstraj



What were the real difficulties?

The primary difficulty was handling the coordinate frame transformations and ensuring the yaw control logic handled the discontinuities at pm pi correctly. Neglecting the angle difference calculation led to inefficient rotation behaviors.

The most significant hurdle after failing to design and tune the controller at the FROST lab in the designated time slots was the "destabilization" of the drone due to VM hardware latency. The default Gazebo step size caused the Simulink controller to miss update windows, leading to erratic behavior. Identifying the need for a **1000 Hz / 0.001s** configuration was the turning point that allowed the controller to work. However if the controller were to be tested with a real drone it would most likely function without much tuning as the sensors and hardware at the FROST lab were well calibrated (most of the time).

We observed that yawing is not decoupled from lift in a quadcopter. Rotating the drone consumes motor headroom and changes the effective thrust vector. Learning to mix a compensation signal into the thrust command was a key takeaway for achieving high-performance flight. This could have been tuned even more to create a smooth Z trajectory but the time constraint led us to be happy with the current performance.

Initially, we neglected the discontinuity at ±π(180°), causing the drone to rotate the long way around the circle (e.g. turning 350 degrees left instead of 10 degrees right).

Implementing the angdiff logic solved this, ensuring efficient heading changes.

5. Conclusion

The group successfully implemented a full control stack for the Crazyflie 2.0. By moving away from pre-packaged blocks and designing the control logic (Rotation matrices, Thrust mixing) manually, we gained a deeper understanding of aerial robotics dynamics. Despite the simulation challenges, the final system demonstrated robust manual control and capable trajectory tracking for complex 3D paths. However, more tuning and real world testing would improve upon this design and its flaws and also reduce any hardware limitations we encountered in our home systems.